LXC — Linux Containers

LXC (англ. **L**inu**x** **C**ontainers) — система виртуализации на уровне операционной системы для запуска нескольких изолированных экземпляров ОС Linux на одном компьютере. LXC не использует виртуальные машины, а создает виртуальное окружение с собственным пространством процессов и сетевым стеком. Все экземпляры LXC используют один экземпляр ядра ОС. Данная система похожа на системы OpenVZ и Linux-VServer для ОС Linux, и на FreeBSD jail и Solaris Containers. LXC основана на технологии ядра Linux под названием cgroups (добавлено в версии ядра 2.6.29). Также используется функциональность изоляции namespaces.

Виртуализация на уровне операционной системы — метод виртуализации, при котором ядро операционной системы поддерживает несколько изолированных экземпляров пространства пользователя, вместо одного. Эти экземпляры (часто называемые контейнерами или зонами) с точки зрения пользователя полностью идентичны реальному серверу. Для систем на базе UNIX, эта технология может рассматриваться как улучшенная реализация механизма chroot. Ядро обеспечивает полную изолированность контейнеров, поэтому программы из разных контейнеров не могут воздействовать друг на друга.

Стефан Грабер (Stéphane Graber) работает в Canonical в Ubuntu Foundations Team. В основном он занимается сетевыми проектами в Ubuntu - IPv6, DNS, LTSP. Стефан с Сержем (Serge Hallyn) являются главными мантейнерами LXC, который зарелизится в версии LXC 1.0 в феврале 2014 года.

Для популяризации LXC Стефан решил опубликовать ряд статей:

- LXC 1.0: Первый Ubuntu контейнер.
- LXC 1.0: Второй контейнер.
- LXC 1.0: Продвинутое использование контейнера.
- LXC 1.0: Более углублённое использование контейнера.
- LXC 1.0: Хранилище контейнеров.
- LXC 1.0: Безопасность.
- LXC 1.0: Непривилегированные контейнеры.
- LXC 1.0: Скрипты и API.
- LXC 1.0: GUI в контейнере.
- LXC 1.0: Решение проблем и отладка.
- VPN в контейнере.

В статьях объясняется, как получить последнюю версию LXC, так как некоторые возможности требуют последней версии, хотя большинство функционала работает и на старых версиях LXC.

Что такое LXC?

LXC - интерфейс в пользовательском пространстве для работы с контейнерами, реализованные в ядре Linux. Мощный API и простые инструменты позволяют линукс пользователям легко создавать и управлять контейнерами.

Проект LXC активно разрабатывается с выпуском промежуточных релизов каждый месяц. Первый стабильный релиз ожидается в феврале 2014 года. В его разработке участвовало 67 разработчиков из разных стран и компаний.

ГитХаб проекта: http://github.com/lxc
Веб сайт: http://linuxcontainers.org
Почтовая рассылка: http://lists.linuxcontainers.org

LXC 1.0

LXC 1.0 - первый, реально стабильный релиз LXC, который будет поддерживаться 5 лет и будет поставляться с Ubuntu 14.04 LTS, выходящая в апреле 2014 года.

LXC 1.0 будет идти с:

•стабильным API.
•набором новых, интересных возможностей, которые будут описаны в следующих статьях.
•поддержкой широкого спектра хостовых и гостевых дистрибутивов, в том числе Android.

Как получить LXC?

Ради справки, стоит знать, что LXC в Ubuntu, начиная с 12.04. Сейчас речь пойдёт о получении более новых версий.

Подразумевается далее по тексту, что вы используете Ubuntu. Для получения последних версий LXC для Ubuntu 12.04, 12.10, 13.04, 13.10, 14.04 можно использовать PPA https://launchpad.net/~ubuntu-lxc/+archive/daily

Если вы по непонятной причине не хотите использовать deb пакеты с дневными билдами и хотите самостоятельно скомпилировать нужное, хотя это не рекомендуется в linux системах с пакетным менеджментом, то на свой страх и риск:

```
git clone git://github.com/lxc/lxc
cd lxc
sh autogen.sh
./configure
make
sudo make install
```

Первый контейнер.

Предположим, что вы так или иначе используете последнюю версию LXC, хотелось бы верить что применяя пакеты deb.

Создаём контейнер **p1**, используя шаблон **ubuntu** и такую же архитектуру и версию как и хост. Использование "-- --help" выведет список всех доступных опций.

```
sudo lxc-create -t ubuntu -n p1
```

Запуск контейнера в фоне.

```
sudo lxc-start -n p1 -d
```

Вход в контейнер одним из различных путей:

•Присоединение к консоли контейнера (ctrl-a + q для отсоединения)

```
sudo lxc-console -n p1
```

•Запуск bash напрямую в контейнере, минуя логин. Требуется ядро >= 3.8.

```
sudo lxc-attach -n p1
```

SSH в контейнер. Логин ubuntu, пароль ubuntu.

```
sudo lxc-info -n p1
ssh ubuntu@IP-из-lxc-info
```

Остановка контейнера одним из различных путей:

•Остановка контейнера из самого контейнера.

```
sudo poweroff
```

•Корректная остановка контейнера из хоста.

sudo lxc-stop -n p1

•Грубое убийство контейнера из хоста.

sudo lxc-stop -n p1 -k

Вот и готов первый контейнер. Как и обещано, в Ubuntu - всё просто! Ядра обладают поддержкой всего что нужно для LXC и контейнер использует bridge и DHCP по умолчанию. Естественно, всё можно изменить и настроить под свои нужды, но это в следующих статьях.

Больше шаблонов.

В статье LXC 1.0: Первый Ubuntu контейнер легко и просто создали первый контейнер **p1** с помощью шаблона по умолчанию **ubuntu**. Но LXC поддерживает больше систем, чем просто Ubuntu. На данный момент поддерживаются Alpine Linux, Alt Linux, Arch Linux, busybox, CentOS, Cirros, Debian, Fedora, OpenMandriva, OpenSUSE, Oracle, Plamo, sshd, Ubuntu Cloud и Ubuntu.

Шаблоны можно найти в /usr/share/lxc/templates. Все шаблоны обладают дополнительными опциями, которые можно узнать по ключу --help после вызова lxc-create. Только нужно применять -- , чтобы разделять опции lxc-create и опции шаблона.

Написание дополнительных шаблонов не сложно. Это просто исполняемые файлы, часто, но не обязательно, это shell скрипты, которые понимают набор стандартных аргументов и выполняют rootfs в нужном пути.

В связи с недостающими инструментами не все дистрибутивы Linux могут самозагружать (Bootstrapping) любой дистрибутив. Разработчики LXC заинтересованы в улучшении ситуации, даже если используются странные трюки как в Fedora, так что если есть не работающая комбинация хост + гость, то патчи приветствуются!

Но хватит болтать, давайте двигаться дальше.
sudo lxc-create -t oracle -n p2 -- -a i386

Но скорее всего установка провалится, так как нет rpm и yum. Установите их и попробуйте снова. После установки нужных пакетов rpm и после создания контейнера, можно запустить его:
sudo lxc-start -n p2

Приглашение в систему просто просит ввести - логин root и пароль root.

Контейнер запущен без ключа -d поэтому для возврата в хост-shell придётся завершить работу контейнера. Нельзя отсоединиться (detach) от контейнера, который был запущен не в фоне с ключом -d.

Теперь стоит задаться вопросом - почему два шаблона Убунту? Обычный шаблон ubuntu, который использовался до сих пор, создавал контейнер "линукс с нуля" через debootstrap. Шаблон ubuntu-cloud скачивает подготовленный облачный образ и запускает его. Образ включает в себя cloud-init и поддерживает стандартные облачные метаданные.

Какой шаблон лучше использовать - дело вкуса. Шаблон ubuntu может быстро работать, если репозитория расположены на быстрых локальных зеркалах. И главное, что шаблоны используют кэш. Для данной архитектуры первоначальное создание контейнера может быть медленным, но последующие будут использовать локальную копию из кэша и всё будет происходить быстрее.

Автозагрузка.

Как сделать, чтобы контейнер автоматически стартовал при загрузке хоста? Долгое время в Убунту и других дистрибутивах использовались init скрипты и симлинки в /etc/, но совсем недавно была реализована автозагрузка в upstream'e.

Как реализована сейчас автозагрузка контейнера? Каждый контейнер описывается своим конфигурационным файлом в /var/lib/lxc/имя-контейнера/config, в котором можно видеть строки
ключ = значение
Все значения можно прочесть в lxc.conf(5).

Значения, связанные с автостартом контейнера:

•lxc.start.auto = 0 (выключено) или 1 (включено)
•lxc.start.delay = 0 (задержка в секундах до автостарта контейнера)
•lxc.start.order = 0 (приоритет контейнера, чем выше значение, тем раньше стартует контейнер)
•lxc.group = group1,group2,group3,… (членами каких групп является контейнер)

Когда стартует хост-машина, init скрипт lxc-autostart запустит все контейнеры, у которых не указана группа, в правильном порядке и через определённое ими время, но в конфиге контейнера указано lxc.start.auto=1.

Чтобы показать работу механизма автостарта. Поправим /var/lib/lxc/p1/config и внесём:
lxc.start.auto = 1
lxc.group = ubuntu

В /var/lib/lxc/p2/config внесём:
lxc.start.auto = 1
lxc.start.delay = 5
lxc.start.order = 100

После перезагрузки хост-машины, только контейнер p2 стартанёт автоматически, так как он указан без группы. lxc.start.order не играет сейчас никакой роли, так как стартующий контейнер сейчас один, но будет 5 секундная задержка перед стартом p2.

Можно проверить, какие контейнеры автоматически стартуют, с помощью sudo lxc-ls --fancy

```
NAME    STATE    IPV4        IPV6                                      AUTOSTART
----------------------------------------------------------------------------------
p1      RUNNING  10.0.3.128  2607:f2c0:f00f:2751:216:3eff:feb1:4c7f  YES (ubuntu)
p2      RUNNING  10.0.3.165  2607:f2c0:f00f:2751:216:3eff:fe3a:f1c1  YES
```

Можно с консоли управлять контейнерами, используя lxc-autostart и параметры start, stop, kill, reboot для любого контейнера, имеющего в его конфиге lxc.start.auto=1.

Для примера, команда sudo lxc-autostart -a запустит все контейнеры, имеющие lxc.start.auto=1, игнорируя значение lxc.group. В нашем случае с двумя контейнерами p1 и p2, сначала произойдёт запуск контейнера p2, из-за его приоритета order=100, но с задержкой в 5 секунд. Потом стартанёт p1.

Если нужно перезагрузить все контейнеры из группы Ubuntu, а у нас в примере это только p1, то командуем sudo lxc-autostart -r -g ubuntu

Ключ -L позволит увидеть какие контейнеры это затронет и как будет происходить автозапуск, но в реальности ничего не будет выполнено, что удобно при отладке.

Заморозка контейнеров.

Иногда контейнер активно не используется, а его периодический старт и выключение занимают определённое время. Есть возможность вместо выключения/включения контейнера использовать заморозку.

sudo lxc-freeze -n имя-контейнера замораживает все процессы в контейнере, так что они не тратят такты, отведённые им планировщиком. Однако процессы продолжают существовать и потреблять ОЗУ.

sudo lxc-unfreeze -n имя-контейнера позволит разморозить процессы в нужном вам контейнере.

Сеть.

По умолчанию в Ubuntu выделяется по одному veth устройству на контейнер, который соединён в сетевой мост с lxcbr0 на хосте и на нём запущен dnsmasq dhcp сервер.

Такой схемы обычно хватает большинству, но можно реализовать и более сложное, если это необходимо. Например, несколько сетевых интерфейсов внутри контейнера или проброс физического сетевого интерфейса хоста прямиком в контейнер.

Все сетевые возможности LXC можно узнать в lxc.conf(5), но один пример можно привести.

```
lxc.network.type = veth
lxc.network.hwaddr = 00:16:3e:3a:f1:c1
lxc.network.flags = up
lxc.network.link = lxcbr0
lxc.network.name = eth0

lxc.network.type = veth
lxc.network.link = virbr0
lxc.network.name = virt0

lxc.network.type = phys
lxc.network.link = eth2
lxc.network.name = eth1
```

Контейнер будет иметь 3 интерфейса:

•eth0 (обычный veth) будет соединён в сетевой мост с lxcbr0.
•eth1 - это проброшенный физический интерфейс eth2 хоста. Когда контейнер стартанёт, eth2 исчезнет в хост-машине.
•virt0 ещё одно veth устройство, соединёное в мост с virbr0 хоста.

Последние 2 интерфейса не имеют MAC адреса и сетевых флагов (network.flags), поэтому они будут получать при загрузки данного контейнера всегда разные MAC адреса и данные интерфейсы нужно самим поднимать после старта контейнера.

Присоединение (Attach).

При условии, что используется ядро 3.8 и выше, можно использовать инструмент lxc-attach. Его основной задачей является дать shell внутри запущенного контейнера: sudo lxc-attach -n p1

Можно сразу выполнять команду в контейнере, используя sudo lxc-attach -n p1 -- restart ssh

Более опасный и мощный приём - sudo lxc-attach -n p1 -e -s 'NETWORK|UTSNAME'.

Где,
-e, --elevated-privileges - не сбрасывать привилегии, перед запуском команды внутри контейнера.

Нужно применять с осторожностью.
-s, --namespaces - указать пространства имён, к которым нужно подсоединиться. Разрешены MOUNT, PID, UTSNAME, IPC, USER и NETWORK.

Мы получим shell, говорящий нам root@p1, благодаря UTSNAME. Выполнив ifconfig -a, получим список сетевых интерфейсов контейнера. НО все остальные команды будут "от хоста". Ключ -e не даст ограничениям cgroup, apparmor применяться к любым процессам внутри данного shell. Чем это и опасно! Но время от времени, может быть полезным для запуска программ, расположенных в хост системе, но внутри контейнера с его сетью и пространством имён PID.

Проброс устройств внутрь контейнера.

Вход в контейнер, запуск программ и выход из контейнера это всё замечательно, но что насчёт доступа к устройствам, подключённым к хосту?

По умолчанию, LXC пресекает такой доступ, используя devices cgroup как фильтрующий механизм. Можно изменить конфигурацию контейнера, разрешив доступ к нужным устройствам и перезапустить контейнер.

Но для разовых вещей есть удобный инструмент lxc-device. Для примера, sudo lxc-device add -n p1 /dev/ttyUSB0 /dev/ttyS0 добавит (mknod) /dev/ttyS0 в контейнер с тем же типом, старшим (major), младшим (minor) числами под именем /dev/ttyUSB0 и добавит разрешающие записи cgroup.

Обмен данными с контейнером.

Так как контейнеры обитают на файловой системе хоста, то передать данные можно разными способами.

Первое очевидное решение - перейти в /var/lib/lxc/имя-контейнера/rootfs/

Но иногда этого не достаточно, так как в контейнере могут использоваться и монтироваться файловые системы типа tmpfs. В таких случаях можно применить трюк:
sudo ls -lh /proc/$(sudo lxc-info -n p1 -p -H)/root/run/
Вам отобразят содержимое /run контейнера p1.

Из хоста брать данные у контейнера здорово, но что если нужно из контейнера получать доступ и писать данные в хост?

Допустим, что на хостовой машине есть папка /var/cache/lxc и мы хотим поделиться ею с контейнером p1. Добавим в файл /var/lib/lxc/p1/fstab строку
/var/cache/lxc var/cache/lxc none bind,create=dir

Строка означает, что /var/cache/lxc из хоста монтируется (bind-mount, файловый тип *none* и опция *bind*) в контейнере как /var/cache/lxc (отсутствие лидирующего / делает путь относительно корня контейнера) и создать любые отсутствующие каталоги в контейнере (*create=dir*). После рестарта p1 в его /var/cache/lxc можно увидеть тоже самое что и в хосте. Если нужно дать контейнеру доступ на чтение, то просто добавить к параметрам монтирования флаг **ro** и всё.

Вложенные контейнеры.

Одна из крутых вещей в LXC, хотя не нужная большинству, это поддержка вложенности. То есть вы можете запустить LXC внутри LXC без дополнительных затрат (overhead).

По умолчанию, вложенность заблокирована в Ubuntu. Так как в контейнере нужно разрешить монтирование cgroupfs, которая позволит улизнуть от любых ограничений cgroup, применяемых к данному контейнеру. То есть если вы недоверяете контейнерам, то лучше не использовать вложенность.

Для примера, включим возможность вложенности для контейнера p1. Для этого правим конфигурационный файл /var/lib/lxc/p1/config и добавляем:
lxc.aa_profile = lxc-container-default-with-nesting

После рестарта p1 нужно установить в нём lxc. Очень рекомендуется установить версию lxc такую же как и в хосте, но это не является обязательным.

Внутри контейнера командуем sudo lxc-create -t ubuntu -n p1. Учитывая что до этого связывали кэш lxc /var/cache/lxc, всё должно произойти очень быстро. Внутри контейнера можно запустить вложенный контейнер как обычно.

Если на хостовой машине запустить команду sudo lxc-ls --fancy --nesting, то можно наблюдать вложенность контейнеров.

```
NAME    STATE    IPV4                IPV6   AUTOSTART
-----------------------------------------------------
p1      RUNNING  10.0.3.82, 10.0.4.1 -      NO
 \_ p1  RUNNING  10.0.4.7            -      NO
p2      RUNNING  10.0.3.128          -      NO
```

Нет лимитов на вложенность, но трудно будет кому-то объяснить зачем вы создали контейнер на 10ом уровне 

Сырой (raw) сетевой доступ.

В предыдущей статье LXC 1.0: Второй контейнер упоминалось про прокидывание устройств из хоста внутрь контейнера. Но на примере другого контейнера можно показать работу с сырым сетевым доступом. Данный контейнер создан Стефаном с целью иметь под рукой стандартного клиента для имитации работы в удалённой сети по VPN в изолированной среде, не затрагивая сеть хоста.

Стефан на своём ноутбуке-хосте поднял OpenVPN и получил tap0 (raw ethernet tap device).

Контейнер использует в своём конфиге следующие сетевые настройки:
lxc.network.type = phys
lxc.network.hwaddr = 00:16:3e:c6:0e:04
lxc.network.flags = up
lxc.network.link = tap0
lxc.network.name = eth0

Для тестов, Стефан запускает OpenVPN на своём хосте-ноутбуке и запускает контейнер, который тут же крадёт tap0 и использует его как eth0. Через DHCP контейнер получит IP и будет вести себя как будто он - удалённая физическая машина. Указан MAC адрес для удобства, чтобы не получать каждый раз случайный MAC.

Запуск других архитектур.

По умолчанию LXC позволяет работать с контейнерами тех архитектур, которые поддерживается хостом и это естественно, так как ваш CPU не знает, что делать с инструкциями отличных от его.

Но есть волшебный пакет qemu-user-static, который содержит кучу эмуляторов популярных архитектур: ARM, CRIS, i386, M68k (ColdFire), MicroBlaze, MIPS, PowerPC, SH4, SPARC и x86-64. Шаблон **ubuntu** знает как использовать qemu-user-static, если он установлен, и можно создать контейнер с архитектурой ARM так:
sudo lxc-create -t ubuntu -n p3 -- -a armhf

После довольного долгого bootstrap'а будет создан контейнер p3, который по большей части будет обеспечивать работу Ubuntu на armhf. По большей части потому, что эмуляция QEMU идёт с рядом ограничений. Самое серьёзное ограничение - софт не может использовать системный вызов ptrace() и netlink. Поэтому LXC поставит в контейнер часть софта с архитектурой хоста, а не ту что ему указали. В эту часть софта входят: upstart (из-за ptrace) и mountall, iproute, isc-dhcp-client (из-за netlink).

Хост file /bin/ls

```
/bin/ls: ELF 64-bit LSB  executable, x86-64, version 1 (SYSV), dynamically linked (uses shared libs),
for GNU/Linux 2.6.24, ""BuildID[sha1]"" =e50e0a5dadb8a7f4eaa2fd715cacb9842e157dc7, stripped
```

Запуск контейнера
sudo lxc-start -n p3 -d
sudo lxc-attach -n p3

В контейнере root@p3:/# file /bin/ls

```
/bin/ls: ELF 32-bit LSB  executable, ARM, EABI5 version 1 (SYSV), dynamically linked (uses shared libs),
for GNU/Linux 2.6.32, ""BuildID[sha1]"" =88ff013a8fd9389747fb1fea1c898547fb0f650a, stripped
```

Хуки.

Конфигурация контейнера не всегда может удовлетворить все случаи использования и поэтому введён ряд хуков. Хуки представляют собой исполняемые файлы с полным путём к нему, которые LXC будет вызывать в определённое время в жизненном цикле контейнера. Хукам от LXC передаются полезные переменные окружения, чтобы они легко понимали с каким контейнером их вызвали и что делать.

Список хуков (детали в lxc.conf(5))

•**lxc.hook.pre-start** вызывается ДО завершения любых инициализаций.
•**lxc.hook.pre-mount** вызывается ПОСЛЕ создания пространства имён mount, но ДО монтирования чего-либо.
•**lxc.hook.mount** вызывается ПОСЛЕ монтирования, но ДО pivot_root.
•**lxc.hook.autodev** идентично хуку mount, но вызывается только если используется autodev.
•**lxc.hook.start** вызывается ПЕРЕД непосредственным вызовом /sbin/init.
•**lxc.hook.post-stop** вызывается ПОСЛЕ остановки контейнера.
•**lxc.hook.clone** вызывается когда клонируется контейнер.

Каждая секция в конфигурационном файле, содержащая сетевые настройки может использовать 2 хука:

•**lxc.network.script.up** вызывается в сетевом пространстве имён ПОСЛЕ создания интерфейса.
•**lxc.network.script.down** вызывается в сетевом пространстве имён ДО удаления интерфейса.

Для примера, укажем для контейнера p1 в его конфиге
lxc.hook.pre-start = /var/lib/lxc/p1/pre-start.sh

Создадим исполняемый файл /var/lib/lxc/p1/pre-start.sh

```
#!/bin/sh
echo "arguments: $*" > /tmp/test
echo "environment:" >> /tmp/test
env | grep LXC >> /tmp/test
```

После перезагрузки контейнера p1, должен создаться /tmp/test с содержимым:
arguments: p1 lxc pre-start
environment:
LXC_ROOTFS_MOUNT=/usr/lib/x86_64-linux-gnu/lxc
LXC_CONFIG_FILE=/var/lib/lxc/p1/config
LXC_ROOTFS_PATH=/var/lib/lxc/p1/rootfs
LXC_NAME=p1

Контейнер с Андроидом.

Часто интересуются вопросом, а возможен запуск Android в LXC контейнере? Кратко: **да**. Однако, не всё так просто и смотря что хочется с ним сделать.

Первое что нужно сделать для запуска Андроида, нужны загруженные модули для построения и запуска контейнера. Предположим, что вы создали контейнер **android** с архитектурой armhf и нужно подправить его конфигурационный файл и добавить строки:

```
lxc.rootfs = /var/lib/lxc/android/rootfs
lxc.utsname = armhf

lxc.network.type = none

lxc.devttydir = lxc
lxc.tty = 4
lxc.pts = 1024
lxc.arch = armhf
lxc.cap.drop = mac_admin mac_override
lxc.pivotdir = lxc_putold

lxc.hook.pre-start = /var/lib/lxc/android/pre-start.sh

lxc.aa_profile = unconfined
```

Самое главное тут - хук /var/lib/lxc/android/pre-start.sh. В нём должны быть строки

```
#!/bin/sh
mkdir -p $LXC_ROOTFS_PATH
mount -n -t tmpfs tmpfs $LXC_ROOTFS_PATH
cd $LXC_ROOTFS_PATH
cat /var/lib/lxc/android/initrd.gz | gzip -d | cpio -i
# Create /dev/pts if missing
mkdir -p $LXC_ROOTFS_PATH/dev/pts
```

Когда стартует LXC контейнер Android, его initrd распаковывается в tmpfs (аналог андроидного ramfs). Android Init стартует, монтирует нужные разделы и запускает свои сервисы. Так как мы указали неограниченный (unconfined) AppArmor, то без ограничений cgroup контейнер будет обладать многими правами и даже может привести к краху хост-машины. Нужны знания об устройстве Android и не бояться вносить изменения в init скрипты. Нельзя дать единый рецепт, поскольку многое зависит от: версия Андроида, что вы хотите реализовать, используемые устройства. Но самое главное, что всё реализуемо и можно многое почерпнуть из проекта Ubuntu Touch, где в LXC контейнере работает Андроид над низкосистемными вещами и для использования закрытых видеодрайверов.

Последнее замечание. У Андроид init скрипт НЕ /sbin/init, поэтому запуск контейнера с Android нужно выполнять так: sudo lxc-start -n android -- /init

LXC на Android.

Выше было рассказано, как в Ubuntu хост создать контейнер LXC, а в нём Android. Можно на Андроид хосте создать LXC контейнер и туда посадить Ubuntu. LXC портирована на bionic (Android C library) и, хотя это не полный эквивалент версии с glibc, но достаточно хорошо работает.

К сожалению, из-за низкоуровневых требований LXC (это ведь не игрушка), вы не сможете легко и просто посетить Google PlayStore и найти там LXC или скачать apk файл и установить.

Всегда новый LXC для использования на платформе Android можно найти по адресу: http://qa.linuxcontainers.org/master/current/android-armel/lxc-android.tar.gz

Гарантируется работа с Android >= 4.2, но и старые версии должны работать без проблем. Для того чтобы всё заработало, требуется конфигурация вашего ядра и lxc-checkconfig должен отобразить совместимость с LXC или отсутствие оной. К сожалению, вероятнее всего ответ - нет! Тогда нужны исходники ядра для вашего устройства и, добавив нужные параметры ядра, скомпилировать и обновить ядро.

Это страшно звучит, но на практике всё не так сложно. Особенно, если используется альтернативный ROM типа Cyanogen. После проделанной работы на ядром, нужно просто распаковать скачанный тарбол в корень (/) вашего устройства. Ваш контейнер с Ubuntu на архитектуре ARM нужно скопировать в /data/lxc/containers/имя-контейнера и запустить ./run-lxc lxc-start -n имя-контейнера

Несколько секунд и должно появиться приветствие с запросом логина и пароля.

Хранилища.

LXC поддерживает различные хранилища (storage backends - backingstore). По умолчанию используется **none**, обозначая место контейнера как /var/lib/lxc/имя-контейнера/rootfs/. Но можно переопределить хранилище в утилитах lxc-create или lxc-clone с помощью ключа -B.

Хранилище на базе каталога (none и dir).

Это хранилище используется по умолчанию и контейнер будет находиться в /var/lib/lxc/имя-контейнера/rootfs/. Опция --dir при использовании -B dir позволит указать свой каталог для создания там контейнера.

btrfs.

С этим backingstore LXC создаст новый подтом (subvolume) для контейнера, что позволит легко делать снимки (snapshots) средствами btrfs.

lvm.

LV может быть установлен --lvname (по умолчанию имя контейнера).
VG может быть установлен --vgname (по умолчанию lxc).
Файловая система может быть установлена --fstype (по умолчанию ext4).
Размер может быть установлен --fssize (по умолчанию 1 Гб).
Для использования LVM thinpools --thinpool.

overlayfs.

Используется в основном при клонировании контейнера, создавая копию другого контейнера и сохраняя изменения от оригинала в виде overlay.

Создавая контейнер через lxc-create, мы получим обычный контейнер, но изменения после первоначального образа будут сохраняться в delta0 каталоге.

zfs.

Подобно btrfs, использование zfs позволит применять возможности, но уже ZFS: подтом (subvolume) для контейнера, быстрое создание снимков и клонов, более эффективное использование места на диске за счёт дедупликации.

Стандартные пути.

LXC обычно использует:

•/var/lib/lxc/ - дефолтное место для контейнеров.
•/var/lib/lxcsnap/ - дефолтное место снимков.
•/var/cache/lxc/ - дефолтное место для кэша шаблонов.
•$HOME/.local/share/lxc/ - дефолтное место для непривилегированных контейнеров.
•$HOME/.local/share/lxcsnap/ - дефолтное место для непривилегированных снимков.
•$HOME/.cache/lxc/ - дефолтное место для непривилегированного кэша шаблонов.

Дефолтный путь, называемый lxcpath, может быть переопределён в командной строке ключом -P или навсегда через
lxcpath = /новый/путь
в /etc/lxc/lxc.conf или $HOME/.config/lxc/lxc.conf для непривилегированных контейнеров.

Если вы переопределяете lxcpath, то каталог снимков snap добавляется к lxcpath и магическим образом за ним следует. Кэш шаблонов, к сожалению, жёстко вшит и не просто его сменить. Дефолтная конфигурация для всех контейнеров находится в /etc/lxc/default.conf и нет эквивалента для непривилегированных контейнеров. Шаблоны операционных систем хранятся в /usr/share/lxc/templates/

Клонирование контейнеров.

Каждое хранилище по своему облегчает возможности клонирования контейнера.

Создадим копию p1 под именем p4, где -o (--orig) и -n (--new)
sudo lxc-clone -o p1 -n p4

Мы создали простую копию контейнера методом полного копирования файлов, что на некоторых файловых системах может занять приличное время. Допустим, мы не собираемся сильно изменять копию p1, тогда можно использовать клонирование так:
sudo lxc-clone -o p1 -n p1-test -B overlayfs -s

Мы получим клон p1 по имени p1-test, который будет использовать rootfs p1 и малейшие изменения будут сохраняться в каталоге delta0 у p1-test. Ключ -s (--snapshot) позволит использовать возможности создания снимков btrfs, zfs, LVM вместо полного копирования.

Создание снимков.

Рассмотренное выше клонирование является хорошим и удобным инструментом и отлично подходит в среде разработки программистов, когда контейнеры клонируются и уничтожаются за ненадобностью. Но в производстве гораздо полезнее снимки для создания резервных копий или для страхования перед опасными изменениями.

В LXC есть утилита lxc-snapshot, которая позволяет создавать, выводить список, восстанавливать и удалять снимки контейнеров. Тут нужно сделать замечание о том, что только в версии LXC 1.0 стало возможно использовать так же хранилища на основе каталога (none и dir). До версии 1.0 снимок получался пустой.

Представим для примера, что у нас есть контейнер p1-lvm и мы хотим поставить apache2 и нужно подстраховаться. Делаем файл с комментарием и командуем

```
echo "before installing apache2" > snap-comment
sudo lxc-snapshot -n p1-lvm -c snap-comment
```

Можно подвериться и посмотреть - создался ли снимок? sudo lxc-snapshot -n p1-lvm -L -C

Теперь можно смело ставить Апач в контейнере. Если, гипотетически, что-то пошло не так, то можно откатиться назад: sudo lxc-snapshot -n p1-lvm -r snap0 или восстановить снимок как отдельный контейнер: sudo lxc-snapshot -n p1-lvm -r snap0 p1-lvm-snap0.

Новый контейнер p1-lvm-snap0 содержит рабочую копию p1-lvm как это было до изменений.

Когда речь заходит о безопасности использования контейнеров, то люди склонны рассматривать их или небезопасными или безопасными по природе своей. Но реальность не чёрно-белая и LXC поддерживает ряд технологий, которые смягчают большинство опасений.

Самого начала хочется развенчать миф, дескать LXC безопасна только при условии использования привилегированных контейнеров. Стефан Грабер является мантейнером LXC в Ubuntu и даёт гарантии, что дефолтные шаблоны и сама Убунту-хост использует отличные конфигурации cgroup и профиля AppArmor, которые предотвращают все известные атаки на сегодняшний день.

Просто имейте в виду, что если используете непривилегированные контейнеры, не давайте root доступ в контейнере тому, кому не хотели бы дать root доступ к вашему хосту.

Возможности (Capabilities).

Первая фича безопасности основана на поддержке Linux capabilities. Можно указать какие возможности (capabilities) будут сброшены (drop) при старте контейнера ИЛИ указать список разрешённых (keep) возможностей, а все не указанные явно будут сброшены.

Для этих целей есть два параметра:

•lxc.cap.drop
•lxc.cap.keep

Весь список доступных возможностей можно узнать в man 7 capabilities.

Вроде всё звучит радужно, как отличный способ сделать контейнеры безопасными и для определённых случаях это так. Во-первых, вскоре можно заметить, что некоторые сброшенные возможности не практичны. Во-вторых, следует знать, что root в контейнере может вернуть себе утраченные возможности.

В шаблоне Ubuntu по умолчанию используется lxc.cap.drop для сброса таких возможностей как sys_module, mac_admin, mac_override, sys_time. Это предотвращает некоторые известные проблемы при загрузке контейнера.

Контрольные группы.

Контрольные группы интересны тем, что делают массу взаимосвязанных, но довольно разных вещей:

•подсчёт ресурсов.
•квоты на ресурсы.
•ограничения доступа.

Первые два пункта не связаны с безопасностью, хотя квоты позволяют избежать DoS на хост, если установить лимиты на память, CPU и I/O. Последний пункт, касающийся devices cgroup, позволяет указать к каким символьным и блочным устройствам контейнер может получить доступ и что он может сделать с ними. Можно ограничить создание, чтение и запись для каждой комбинации из старшего (major) и младшего (minor) чисел.

В LXC для настройки cgroup используется опция lxc.cgroup.*, которая должна быть определена полностью как
lxc.cgroup.<controller>.<key> = <value>

Для примера можно ограничить использование памяти контейнером p1.
lxc.cgroup.memory.limit_in_bytes = 134217728

Это установит лимит память в 128 Мб, которые указаны в байтах. Альтернативно можно указать лимиты через /sys/fs/cgroup/memory/lxc/p1/memory.limit_in_bytes

Большинство LXC шаблонов устанавливает только несколько записей

```
# Default cgroup limits
lxc.cgroup.devices.deny = a
## Allow any mknod (but not using the node)
lxc.cgroup.devices.allow = c *:* m
lxc.cgroup.devices.allow = b *:* m
## /dev/null and zero
lxc.cgroup.devices.allow = c 1:3 rwm
lxc.cgroup.devices.allow = c 1:5 rwm
## consoles
lxc.cgroup.devices.allow = c 5:0 rwm
lxc.cgroup.devices.allow = c 5:1 rwm
## /dev/{,u}random
lxc.cgroup.devices.allow = c 1:8 rwm
lxc.cgroup.devices.allow = c 1:9 rwm
## /dev/pts/*
lxc.cgroup.devices.allow = c 5:2 rwm
lxc.cgroup.devices.allow = c 136:* rwm
## rtc
lxc.cgroup.devices.allow = c 254:0 rm
## fuse
lxc.cgroup.devices.allow = c 10:229 rwm
## tun
lxc.cgroup.devices.allow = c 10:200 rwm
## full
lxc.cgroup.devices.allow = c 1:7 rwm
## hpet
lxc.cgroup.devices.allow = c 10:228 rwm
## kvm
lxc.cgroup.devices.allow = c 10:232 rwm
```

Такая конфигурация разрешает контейнеру (точнее udev) создавать любое устройство по его желанию (разрешает маска **m**) и блокирует всё остальное по умолчанию (deny = a), если не указано ниже в allow. Обычно этого достаточно для типичной работы контейнера.

AppArmor.

В LXC добавлена поддержка профилей AppArmor. Поддержка AppArmor проста! Есть опция в конфиге контейнера - lxc.aa_profile, которая указывает на профиль AppArmor. Профиль AppArmor предотвращает любую из известных попыток сбежать из контейнера и нанести вред хосту.

На сегодяшний день, Ubuntu обладает 3 профилями:

•**lxc-container-default** - дефолтное значение, если значение lxc.aa_profile не установлено.
•**lxc-container-default-with-nesting** - такое же как дефолтное значение, но разрешается вложенность контейнеров.
•**lxc-container-default-with-mounting** - такое же как дефолтное значение, но разрешается монтирование ФС ext*, xfs и btrfs.
•**unconfined** - специальное значение, которое отключает AppArmor для данного контейнера.

Можно создать свои профили! Поместить их в /etc/apparmor.d/lxc/, перезагрузить sudo /etc/init.d/apparmor reload и указывать имя профиля в опции lxc.aa_profile.

SELinux.

Поддержка SELinux подобна AppArmor'у. Контекст SELinux можно задать с помощью lxc.se_context.

Пример:
lxc.se_context = unconfined_u:unconfined_r:lxc_t:s0-s0:c0.c1023

Как и в случае с AppArmor, LXC переключится на новый контекст перед стартом init в контейнере.

Seccomp.

Seccomp - это фильтры безопасного вычисления (**SEC**ure **COMP**uting mode). Seccomp сравнительно недавно появившийся механизм, который позволяет фильтровать системные вызовы (syscalls). Можно написать свой файл политики для конкретного контейнера и тем самым позволять или запрещать системные вызовы.

Для примера возьмём бесполезный и ограниченный файл политики для seccomp:
1
whitelist
103

Здесь мы разрешаем только системный вызов #103 syslog в контейнере и запрещаем всё остальное. Seccomp является низкоуровневой "защитой" и полезна только в очень специфических случаях использования. Все системные вызовы должны быть адресованы по их ID, а не по их именам, но они меняются при смене архитектуры. Если у вас хост 64 битный, то при загрузке политики seccomp все 32 битные системные вызовы будут запрещены. Нужны универсальные seccomp профили, но пока это не первоочередная задача.

Пользовательское пространство имён. User namespace.

Последнее, но не менее важное, механизм, позволяющий реально сделать контейнеры безопасными. LXC поддерживает user namespace. Более подробно об этой возможности будет описано в следующей статье, но кратко говоря, LXC больше не работает от root'а и злоумышленник, осуществивший побег из контейнера, окажется обычным пользователем в хост-системе. Ну пробил ты головой стенку камеры и что делать в чужой темнице?

Поддержка непривилегированных контейнеров (unprivileged containers), пожалуй, важнейшая новая возможность в LXC 1.0. В предыдущих постах было вскользь упомянуто, что не смотря на раздельные пространства имён (namespace), UID 0 (root) в контейнере по-прежнему

равен UID 0 вне контейнера в хосте. Это означает, что если каким-то образом получить доступ к ресурсам хоста через proc, sys или системные вызовы (syscalls), то потенциально можно сбежать из контейнера и стать root'ом в хосте.

Для избежания такой ситуации и были разработаны пользовательские пространства имён (user namespaces). Их создание и реализация плод многолетних усилий по продвижению патчей, необходимых в ядре. И вот в ядре linux 3.12 окончательно появилось всё необходимое, что позволяет обезопасить LXC контейнеры через user namespaces.

И как же работают эти user namespaces? Говоря упрощённо, каждому пользователю, который будет использовать контейнеры в системе, присваивается диапазон неиспользуемых UID и GID, а в идеале - все 65536. Затем эти UID и GID с помощью утилит newuidmap и newgidmap отображаются (map) из этих UID и GID в виртуальные subUID и subGID для пользовательского пространства имён.

Можно для примера создать контейнер и указать 4 поля (1 поле - **u** или **g**, 2 поле - первый UID в контейнере, 3 поле - первый UID в хосте, 4 поле - диапазон):
lxc.id_map = u 0 100000 65536
lxc.id_map = g 0 100000 65536

Это будет означать, что есть одна карта UID и одна карта GID у моего контейнера, которая UID/GID контейнера от 0 до 65536 ставит в соответствие с UID/GID хоста от 100000 до 165536 (100000+65536). Чтобы это сопоставление работало, нужно указать пользователю (в примере stgraber учётная запись на ноутбуке Стефана Грабера) эти диапазоны:

В файле /etc/subuid - stgraber:100000:65536
В файле /etc/subgid - stgraber:100000:65536

LXC обновлён и все его инструменты умеют работать с непривилегированными контейнерами.

Стандартные пути имеют свои "непривилегированные" эквиваленты:

•/etc/lxc/lxc.conf => ~/.config/lxc/lxc.conf
•/etc/lxc/default.conf => ~/.config/lxc/default.conf
•/var/lib/lxc => ~/.local/share/lxc
•/var/lib/lxcsnaps => ~/.local/share/lxcsnaps
•/var/cache/lxc => ~/.cache/lxc

Таким образом, пользователь может создавать контейнеры, в которых он будет обладать привилегиями root'а, но это не будет давать ему никаких супер прав в системе хоста.

Правда остаются вопросы о таких системных вещах как сетевые устройства и изменение конфигурации сетевого моста. Пользователь должен иметь возможность работать с ними и изменять их, но не обладать правами root'а в хост-машине. Для реализации задуманного написана утилита lxc-user-nic, которая единственная из всех утилит LXC 1.0 обладает SETUID битом. Задача lxc-user-nic проста: она парсит конфигурационный файл и на основе его создаёт сетевые устройства для пользователя. Для предотвращения злоупотреблений, можно ограничить количество устройств и запросы на соединения в сетевой мост.

Для примера, в файле /etc/lxc/lxc-usernet можно задать ограничение для пользователя stgraber:
stgraber veth lxcbr0 10

Эта запись разрешает создать 10 устройств типа veth и соединить их сетевым мостом с lxcbr0. Как будут созданы veth устройства - решать вам. Все 10 в одном контейнере (не имеет смысла из-за сетевого моста) или по одному veth в каждом из 10 контейнеров. Ограничение только в количестве, нет ни каких ограничений от LXC, кроме лимитов самого ядра linux.

Благодаря полной поддержке и реализации user namespaces в ядре и единственной setuid утилите, реализована поддержка непривилегированных контейнеров.

Требования к системе.

На дату 28 января 2014 года, требования для нормальной работы непривилегированных контейнеров следующие:

•Ядро linux >= 3.13.
•Включенная поддержка User namespaces в ядре.
•Последняя версия shadow с поддержкой subuid/subgid.
•Cgroups с контролем отдельных пользователей на всех контроллерах.
•LXC 1.0 beta2 или новее.
•Версия PAM (**P**luggable **A**uthentication **M**odules) с патчем к loginuid.

Всем этим требования точно удовлетворяет Ubuntu 14.04 Trusty Tahr, которая пока находится в разработке, обновлённая на дату 28 января 2014 или актуальнее.

Предварительно созданные контейнеры LXC.

Пользовательские пространства имён (user namespaces) для создания непривилегированных контейнеров идут с рядом ограничений. В user namespaces вы не сможете использовать mknod для создания блочных или символьных устройств, так как позволять такое, значит разрешать доступ к хосту. Так же нельзя использовать монтирование ext разделов и mount -t loop, даже если дан доступ к блочному устройству.

Эти ограничения, естественно, не конец света, но в повседневном использовании станут большой проблемой при bootstrap'e контейнера и вызове утилит debootstrap, yum и других, которые точно захотят выполнить эти запрещённые действия и получат отлуп.

Некоторые шаблоны операционных систем были подправлены, чтобы обойти часть ограничений, если пользователь захочет использовать на своём компьютере именно непривилегированные контейнеры. Но проект LXC исходит из того, что пользователь не должен обладать знаниями создателей дистрибутивов Linux и поэтому предложено более простое решение.

Написан новый шаблон **download**, который вместо того чтобы пытаться создать на локальной машине образ нужной системы, просто связывается с сервером, который хранит ежедневные подготовленные rootfs и конфигурации большинства шаблонов операционных систем.

Образы делает Jenkins сервер, используя несколько машин. Сам процесс сборки очень прост. С базовым chroot и стандартными шаблонами запускается сборка с нужным релизом и архитектурой. Получившийся rootfs сжимается и вместе с базовым конфигом и метаданными (время устаревания, файлы шаблона, ...) сохраняются на сервере, подписываются GPG ключом и публикуются в веб.

Со стороны клиента нужно всего лишь соединиться по https (сервер поддерживает DNSSEC и IPv6), взять подписанный индекс всех доступных образов, проверить наличие нужного дистрибутива, релиза и архитектуры. Затем можно скачать нужное, проверить криптографическую подпись и сохранить к себе в локальный кэш. Любое создание контейнера использует данный кэш, но только пока не истечёт указанный период, после чего будет скачана новая копия с сервера.

Текущий список образов, которых можно получить с ключом --list

```
---
DIST      RELEASE   ARCH    VARIANT    BUILD
---
debian    wheezy    amd64   default    20140116_22:43
debian    wheezy    armel   default    20140116_22:43
debian    wheezy    armhf   default    20140116_22:43
```

```
debian    wheezy    i386    default    20140116_22:43
debian    jessie    amd64   default    20140116_22:43
debian    jessie    armel   default    20140116_22:43
debian    jessie    armhf   default    20140116_22:43
debian    jessie    i386    default    20140116_22:43
debian    sid       amd64   default    20140116_22:43
debian    sid       armel   default    20140116_22:43
debian    sid       armhf   default    20140116_22:43
debian    sid       i386    default    20140116_22:43
oracle    6.5       amd64   default    20140117_11:41
oracle    6.5       i386    default    20140117_11:41
plamo     5.x       amd64   default    20140116_21:37
plamo     5.x       i386    default    20140116_21:37
ubuntu    lucid     amd64   default    20140117_03:50
ubuntu    lucid     i386    default    20140117_03:50
ubuntu    precise   amd64   default    20140117_03:50
ubuntu    precise   armel   default    20140117_03:50
ubuntu    precise   armhf   default    20140117_03:50
ubuntu    precise   i386    default    20140117_03:50
ubuntu    quantal   amd64   default    20140117_03:50
ubuntu    quantal   armel   default    20140117_03:50
ubuntu    quantal   armhf   default    20140117_03:50
ubuntu    quantal   i386    default    20140117_03:50
ubuntu    raring    amd64   default    20140117_03:50
ubuntu    raring    armhf   default    20140117_03:50
ubuntu    raring    i386    default    20140117_03:50
ubuntu    saucy     amd64   default    20140117_03:50
ubuntu    saucy     armhf   default    20140117_03:50
ubuntu    saucy     i386    default    20140117_03:50
ubuntu    trusty    amd64   default    20140117_03:50
ubuntu    trusty    armhf   default    20140117_03:50
ubuntu    trusty    i386    default    20140117_03:50
```

Шаблоны написаны и тщательно протестированы для работы на любой системе, где есть POSIX совместимый shell и wget. GPG рекомендуется, но можно отключить на свой страх и риск. Некоторые шаблоны могут повторно использоваться на вашем сервере для создания контейнеров из центрального ресурса, пользуясь механизмом истечения времени, чтобы автоматически предлагать обновлённые шаблоны.

В то время как данные шаблоны призваны помочь в создании непривилегированных контейнеров с их ограничениями, они прекрасно работают с системными контейнерами. Так что даже в системах, которые не поддерживают непривилегированные контейнеры, смело можно делать:
lxc-create -t download -n p1 -- -d ubuntu -r trusty -a amd64

И данная команда даст вам последнюю 64 битную сборку Ubuntu 14.04 Trusty Tahr.

Использование непривилегированных контейнеров.

Описание ограничений закончилось и можно наконец-то приступить к использованию непривилегированных контейнеров. Напоминаю, что сейчас всем условиям удовлетворяет лишь актуальная, ещё разрабатываемая, Ubuntu 14.04. Поэтому сначала обновитесь и доставьте нужное, если не установлено.

sudo apt-get update
sudo apt-get dist-upgrade
sudo apt-get install lxc systemd-services uidmap

Временный обход, пока мы ждём нового менеджера cgroup в Ubuntu, заставляет нас добавить в файл /etc/init/lxc-unpriv-cgroup.conf строки

```
start on starting systemd-logind and started cgroup-lite
script
    set +e
    echo 1 > /sys/fs/cgroup/memory/memory.use_hierarchy
    for entry in /sys/fs/cgroup/*/cgroup.clone_children; do
        echo 1 > $entry
    done
    exit 0
end script
```

Этот трюк необходим, так как logind не конфигурирует нужные для LXC use_hierarchy и clone_children. После этого необходим рестарт вашего компьютера для правильной конфигурации cgroup.

Нужно присвоить себе набор UID и GID.

sudo usermod --add-subuids 100000-165536 $USER
sudo usermod --add-subgids 100000-165536 $USER

Создаём ~/.config/lxc/default.conf с содержимым:
lxc.network.type = veth
lxc.network.link = lxcbr0
lxc.network.flags = up
lxc.network.hwaddr = 00:16:3e:xx:xx:xx
lxc.id_map = u 0 100000 65536
lxc.id_map = g 0 100000 65536

Любой **x** в поле hwaddr разрешает LXC генерировать случайное, но правильное значение для MAC адреса. В файле /etc/lxc/lxc-usernet вы как администратор системы должны разрешить пользователю (наверное себе) использовать сетевые интерфейсы и возможность сетевого моста с физической картой хоста, если используется именно этот тип соединения:
ваш-аккаунт veth lxcbr0 10

Всё что нужно - сделано! Теперь создаём первый непривилегированный контейнер:
lxc-create -t download -n p1 -- -d ubuntu -r trusty -a amd64

Должно выводиться что-то типа

```
Setting up the GPG keyring
Downloading the image index
Downloading the rootfs
Downloading the metadata
The image cache is now ready
Unpacking the rootfs
---
You just created an Ubuntu container (release=trusty, arch=amd64).
The default username/password is: ubuntu / ubuntu
To gain root privileges, please use sudo.
```

После успешного создания можно запускать свой контейнер от обычной учётной записи.

```
$ lxc-start -n p1 -d
$ lxc-ls --fancy
NAME  STATE    IPV4     IPV6     AUTOSTART
----------------------------------------
p1    RUNNING  UNKNOWN  UNKNOWN  NO
```

Работает! Контейнер **p1** - RUNNING! Теперь можно получить консоль запущенного непривилегированного контейнера p1 с помощью lxc-console или через SSH, узнав IP адрес через ARP таблицу arp -n. Как вы могли уже заметить, IP адрес в статусе UNKNOWN, потому что **в настоящее время LXC не может присоединить (attach) непривилегированные контейнеры к пространству имён.** Это так же означает, что некоторые поля lxc-info будут пустыми и нельзя использовать lxc-attach. Сейчас разработчики ищут пути, чтобы реализовать нужное в ближайшем будущем.

Так же есть несколько проблем с управлением заданиями в ядре и с PAM, что приводит иногда к вызову lxc-start с довольно странной консолью, где sudo может не работать.

SSH может не работать на некоторых дистрибутивах. Патч уже послан в апстрим, но он, к сожалению, не охватывает все проблемные случаи и, самое главное, его ещё нет во всех релизах.

Довольно много улучшений с непривилегированными контейнерами не попадут в релиз LXC 1.0 в феврале 2014 года. Естественно, пока рано ожидать многого от непривилегированных контейнеров, но сделан огромный шаг по сравнению что было раньше. Сделан хороший задел на будущее, в котором можно будет использовать непривилегированные контейнеры в ряде интересных случаев.

API.

Первая версия liblxc появилась в LXC 0.9, но в экспериментальном состоянии. В LXC 1.0 будет гораздо полнее API, который охватывает все возможности LXC. Даже инструменты lxc-* теперь используют вызовы API, а не прямые вызовы внутренних функций.

API идёт с рядом тестов, которые являются неотъемлемой частью цикла разработки. Кто не хочет писать на языке C, то есть привязки (bindings) для языков LUA, Python3, Go и Ruby.

Документация по API доступна по адресу:
https://qa.linuxcontainers.org/master/current/doc/api/

Данная документация не есть верх совершенства, не хватает примеров, особенно для привязок, но документация покрывает все функции API и это главное. Любая помощь по улучшению документации API только приветствуется.

Основа.

Лучше начать с очень простого примера на языке C, используя LXC API. В примере создаётся структура *apicontainer*, корневая файловая система (rootfs), используя шаблон *download*. Затем контейнер запускается, печатается его состояние, PID и производится попытка его корректно остановить до уничтожения.

```
#include <stdio.h>
#include <lxc/lxccontainer.h>

int main() {
```

```
    struct lxc_container *c;
    int ret = 1;

    /* Настройка структуры */
    c = lxc_container_new("apicontainer", NULL);
    if (!c) {
        fprintf(stderr, "Failed to setup lxc_container struct\n");
        goto out;
    }

    if (c->is_defined(c)) {
        fprintf(stderr, "Container already exists\n");
        goto out;
    }

    /* Создание контейнера */
    if (!c->createl(c, "download", NULL, NULL, LXC_CREATE_QUIET,
                    "-d", "ubuntu", "-r", "trusty", "-a", "i386", NULL)) {
        fprintf(stderr, "Failed to create container rootfs\n");
        goto out;
    }

    /* Старт контейнера */
    if (!c->start(c, 0, NULL)) {
        fprintf(stderr, "Failed to start the container\n");
        goto out;
    }

    /* Запрос информации */
    printf("Container state: %s\n", c->state(c));
    printf("Container PID: %d\n", c->init_pid(c));

    /* Остановка контейнера */
    if (!c->shutdown(c, 30)) {
        printf("Failed to cleanly shutdown the container, forcing.\n");
        if (!c->stop(c)) {
            fprintf(stderr, "Failed to kill the container.\n");
            goto out;
        }
    }

    /* Уничтожение контейнера */
    if (!c->destroy(c)) {
        fprintf(stderr, "Failed to destroy the container.\n");
        goto out;
    }

    ret = 0;
out:
    lxc_container_put(c);
    return ret;
}
```

Как видно из примера, работать с API LXC в языке программирования C не сложно. Большинство функций достаточно понятны и проверка ошибок проста, так как возвращаемое значение булево, легко его проверить и вывести ошибку на stderr, если это нужно.

Примеры на Python 3.

Возможно, управлять контейнерами через API вы захотите на скриптовых языках. Попробуем предыдущий пример написать, используя ЯП Python 3.

```
import lxc
import sys

# Настройка объекта
c = lxc.Container("apicontainer")
if c.defined:
    print("Container already exists", file=sys.stderr)
    sys.exit(1)

# Создание rootfs
if not c.create("download", lxc.LXC_CREATE_QUIET, {"dist": "ubuntu",
                                                   "release": "trusty",
                                                   "arch": "i386"}):
    print("Failed to create the container rootfs", file=sys.stderr)
    sys.exit(1)

# Старт контейнера
if not c.start():
    print("Failed to start the container", file=sys.stderr)
    sys.exit(1)

# Остановка контейнера
print("Container state: %s" % c.state)
print("Container PID: %s" % c.init_pid)

# Остановка контейнера
if not c.shutdown(30):
    print("Failed to cleanly shutdown the container, forcing.")
    if not c.stop():
        print("Failed to kill the container", file=sys.stderr)
        sys.exit(1)

# Уничтожение контейнера
if not c.destroy():
    print("Failed to destroy the container.", file=sys.stderr)
    sys.exit(1)
```

Для данного примера код на Питоне 3 получился нисколько не проще, чем на C. Но что, если сделать пример чуть более функциональнее? Перебрать имена существующих контейнеров и запустить их, если они не запущены. Подождать когда они запустятся и станут доступны по сети, приказать им обновиться и остановить их.

```
import lxc
import sys
```

```
for container in lxc.list_containers(as_object=True):
     # Старт контейнера (если он не запущен)
     started=False
     if not container.running:
          if not container.start():
               continue
          started=True

     if not container.state == "RUNNING":
          continue

     # ждать соединения с контейнером
     if not container.get_ips(timeout=30):
          continue

     # выполнить команду обновления
     container.attach_wait(lxc.attach_run_command,
                           ["apt-get", "update"])
     container.attach_wait(lxc.attach_run_command,
                           ["apt-get", "dist-upgrade", "-y"])

     # завершить работу контейнера
     if started:
          if not container.shutdown(30):
               container.stop()
```

Наиболее интересное в примере это attach_wait, благодаря которой можно вызвать в пространстве имён контейнера нужную команду. Приведённый ниже пример это ясно доказывает. Если пример сохранить под именем lxc-api.py

```
import lxc

c = lxc.Container("p1")
if not c.running:
     c.start()

def print_hostname():
     with open("/etc/hostname", "r") as fd:
          print("Hostname: %s" % fd.read().strip())

# первый запуск в хостовой системе
print_hostname()

# запуск в контейнере - гостевая система
c.attach_wait(print_hostname)

if not c.shutdown(30):
     c.stop()
```

и вызвать python3 lxc-api.py, то можно будет увидеть что-то типа
Hostname: castiana
Hostname: p1

Функции, которые даёт API, нужно "умножить" на доступные флаги типа LXC_ATTACH_* в C API, позволяя контролировать подключаемые пространства имён, AppArmor, обход ограничений cgroups и так далее. Такая гибкость позволит вам реализовать любые задумки по автоматизации ваших контейнеров. API так же позволяет управлять клонированием контейнера и созданием снимков.

```
import lxc
import os
import sys

if not os.geteuid() == 0:
    print("The use of overlayfs requires privileged containers.")
    sys.exit(1)

# Create a base container (if missing) using an Ubuntu 14.04 image
base = lxc.Container("base")
if not base.defined:
    base.create("download", lxc.LXC_CREATE_QUIET, {"dist": "ubuntu",
                                                   "release": "precise",
                                                   "arch": "i386"})

    # Customize it a bit
    base.start()
    base.get_ips(timeout=30)
    base.attach_wait(lxc.attach_run_command, ["apt-get", "update"])
    base.attach_wait(lxc.attach_run_command, ["apt-get", "dist-upgrade", "-y"])

    if not base.shutdown(30):
        base.stop()

# Clone it as web (if not already existing)
web = lxc.Container("web")
if not web.defined:
    # Clone base using an overlayfs overlay
    web = base.clone("web", bdevtype="overlayfs",
                     flags=lxc.LXC_CLONE_SNAPSHOT)

    # Install apache
    web.start()
    web.get_ips(timeout=30)
    web.attach_wait(lxc.attach_run_command, ["apt-get", "update"])
    web.attach_wait(lxc.attach_run_command, ["apt-get", "install",
                                             "apache2", "-y"])

    if not web.shutdown(30):
        web.stop()

# Create a website container based on the web container
mysite = web.clone("mysite", bdevtype="overlayfs",
```

```
                    flags=lxc.LXC_CLONE_SNAPSHOT)
mysite.start()
ips = mysite.get_ips(family="inet", timeout=30)
if ips:
    print("Website running at: http://%s" % ips[0])
else:
    if not mysite.shutdown(30):
        mysite.stop()
```

Пример создаёт базовый контейнер с именем base с помощью шаблона download, клонирует его с помощью overlayfs под именем web, устанавливает apache2 и клонирует снова под именем mysite.

Заметки.

Данная статья подразумевает использование непривилегированных контейнеров. Это требование не является чем-то сложным, но очень важнó для приложений с графическим интерфейсом.

Предположим, что мы хотим запустить Google Chrome с плагинами Google Talk (Hangout) и Adobe Flash в контейнере, чтобы не рисковать хостовой машиной.

Причины запуска приложений в изолированных контейнерах:

•Есть бинарник, но нет его исходника.
•Внешние не-Ubuntu репозитория, которые могут после добавления пропихнуть в систему всё что пожелают. Только грамотный apt pining может уменьшить риск.
•Софт устанавливает своё задание для Cron, которое добавляет удалённые вами репозитория.
•Софт использует setuid бинарник для создания своей песочницы.
•Желание отделить рабочую среду от программ, страдающих проблемами безопасности, такого как Adobe Flash, который ещё популярен в web.

Ниже будут описаны шаги, которые улучшают безопасность хостовой системы, НО до идеала далеко, так как, чтобы GUI приложения работали корректно, они должны получить доступ к:

•**pulseaudio**: возможна запись звуков.
•**X11**: возможен перехват клавиш или создание скриншотов экрана.
•**dri/snd** устройства: с помощью их умелый злоумышленник может придумать что-нибудь зловредное.

Запуск Google Chrome в контейнере.

Идея проста. Берём для контейнера операционную систему Ubuntu 12.04 i386, которая стабильна и хорошо поддерживается многими.

lxc-create -t download -n precise-gui -- -d ubuntu -r precise -a i386

Не забываем, что речь идёт не о системных контейнерах и поэтому в ~/.local/share/lxc/precise-gui/config внесём строки, заменив USERNAME на свой аккаунт.

```
lxc.mount.entry = /dev/dri dev/dri none bind,optional,create=dir
lxc.mount.entry = /dev/snd dev/snd none bind,optional,create=dir
lxc.mount.entry = /tmp/.X11-unix tmp/.X11-unix none bind,optional,create=dir
lxc.mount.entry = /dev/video0 dev/video0 none bind,optional,create=file
lxc.hook.pre-start = /home/USERNAME/.local/share/lxc/precise-gui/setup-pulse.sh
```

В конфиге у вас будет что-то подобное

```
lxc.id_map = u 0 100000 65536
lxc.id_map = g 0 100000 65536
```

Нужно заменить на строки, подразумевая что ваш UID=1000 и GID=1000.
Команда **id** подскажет, если забыли.

```
lxc.id_map = u 0 100000 1000
lxc.id_map = g 0 100000 1000
lxc.id_map = u 1000 1000 1
lxc.id_map = g 1000 1000 1
lxc.id_map = u 1001 101001 64535
lxc.id_map = g 1001 101001 64535
```

Эти отображения означают, что контейнер будет обладать UID/GID, начиная с 0 до 65535. Эти UID/GID сопоставляются с хостовыми - 100000 до 165535, за исключением UID/GID 1000. Этот трюк позволит пользователю в контейнере получить доступ к X, pulseaudio и устройствам DRI/snd, как пользователь в хосте, что облегчает задачу.

Создаём исполняемый скрипт ~/.local/share/lxc/precise-gui/setup-pulse.sh

```
#!/bin/sh
PULSE_PATH=$LXC_ROOTFS_PATH/home/ubuntu/.pulse_socket
if [ ! -e "$PULSE_PATH" ] || [ -z "$(lsof -n $PULSE_PATH 2>&1)" ]; then
pactl load-module module-native-protocol-unix auth-anonymous=1 \
socket=$PULSE_PATH
fi
```

Если скрипт будет не исполняемым, то LXC проигнорирует его. Данный скрипт связывает pulseaudio в хостовой системе с /home/ubuntu/.pulse_socket в контейнере.

Исправим права доступа в домашней директории пользователя внутри контейнера, подразумевая что ваш UID=1000 и GID=1000.
sudo chown -R 1000:1000 ~/.local/share/lxc/precise-gui/rootfs/home/ubuntu

Контейнер готов, пора его запускать и ставить нужное.

```
lxc-start -n precise-gui -d
lxc-attach -n precise-gui -- umount /tmp/.X11-unix
lxc-attach -n precise-gui -- apt-get update
lxc-attach -n precise-gui -- apt-get dist-upgrade -y
lxc-attach -n precise-gui -- apt-get install wget ubuntu-artwork ca-certificates -y
lxc-attach -n precise-gui -- wget https://dl.google.com/linux/direct/google-chrome-
stable_current_i386.deb -O /tmp/chrome.deb
lxc-attach -n precise-gui -- wget https://dl.google.com/linux/direct/google-talkplugin_current_i386.deb
-O /tmp/talk.deb
lxc-attach -n precise-gui -- dpkg -i /tmp/chrome.deb /tmp/talk.deb
lxc-attach -n precise-gui -- apt-get -f install -y
lxc-stop -n precise-gui
```

На данном этапе, всё что нужно у нас есть, но, чтобы в будущем работа с контейнером была проще, можно создать исполняемый скрипт **start-chrome** и поместить его рядом с **setup-pulse.sh**.

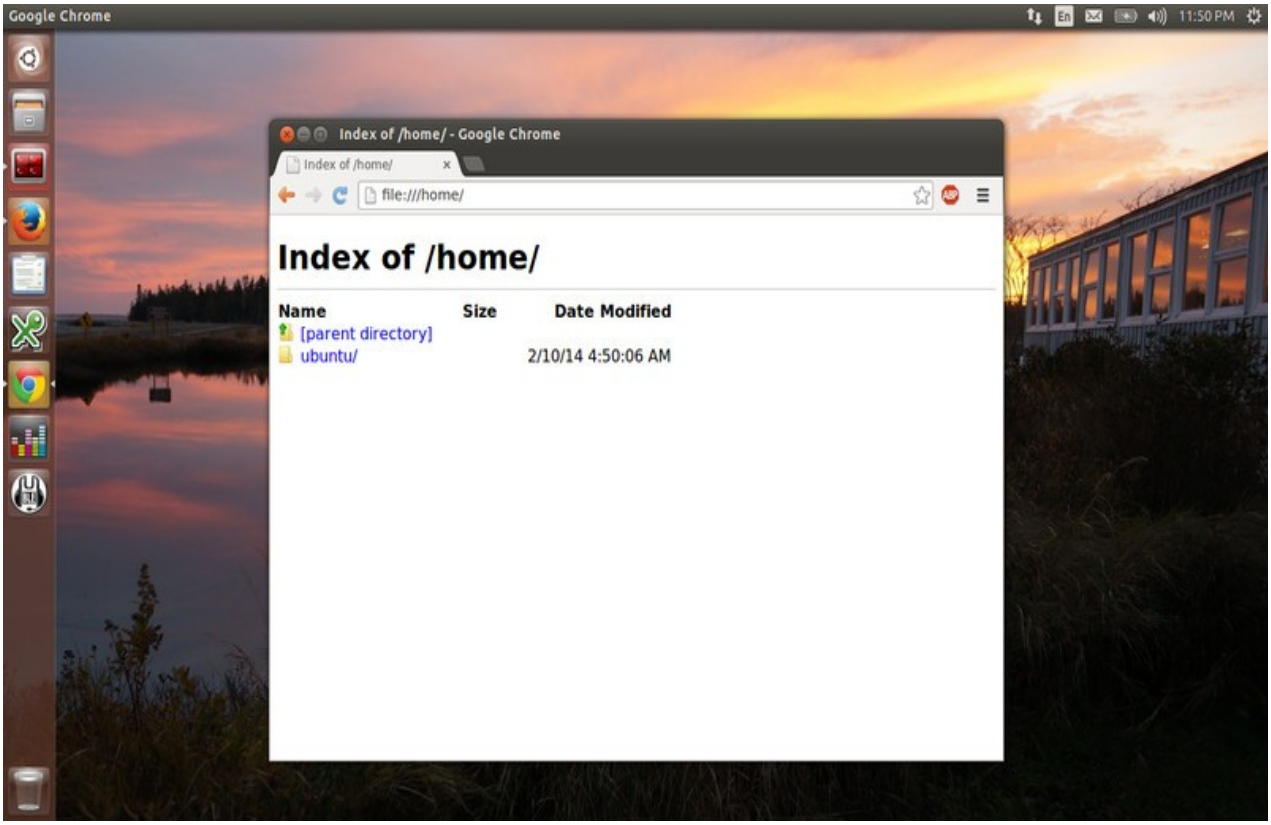
```
#!/bin/sh
CONTAINER=precise-gui
```

```
CMD_LINE="google-chrome --disable-setuid-sandbox $*"
STARTED=false
if ! lxc-wait -n $CONTAINER -s RUNNING -t 0; then
     lxc-start -n $CONTAINER -d
     lxc-wait -n $CONTAINER -s RUNNING
     STARTED=true
fi
PULSE_SOCKET=/home/ubuntu/.pulse_socket
lxc-attach --clear-env -n $CONTAINER -- sudo -u ubuntu -i \
     env DISPLAY=$DISPLAY PULSE_SERVER=$PULSE_SOCKET $CMD_LINE
if [ "$STARTED" = "true" ]; then
     lxc-stop -n $CONTAINER -t 10
fi
```

Скрипт start-chrome проверяет - работает ли контейнер? Если нет, то запускает его. Запускает google-chrome в правильном окружении, отключая его собственную песочницу.

Последний, полирующий штрих - создаём ярлык запуска Google Chrome в контейнере. Замените USERNAME на свой аккаунт.

```
[Desktop Entry]
Version=1.0
Name=Google Chrome
Comment=Access the Internet
Exec=/home/USERNAME/.local/share/lxc/precise-gui/start-chrome %U
Icon=/home/USERNAME/.local/share/lxc/precise-
gui/rootfs/opt/google/chrome/product_logo_256.png
Type=Application
Categories=Network;WebBrowser;
```



У вас будет ярлык для вашей среды, по щелчку которого запускается Хром в контейнере. Можно добавить функционала в виде симлинков к папке Downloads (Закачки) или заюзать bind-mount, чтобы быстро получать доступ к скачанным файлам.

Созданный precise-gui под Google Chrome можно использовать и для запуска других программ, для примера Skype. Или создать под другие программы - другие контейнеры, подобные precise-gui.

Журналирование.

У большинства LXC команд есть два параметра:

•**-o**, --logfile=FILE: указать файл журнала (по дефолту stder)
•**-l**, --logpriority=LEVEL: приоритет журнала (умолчание соответствует ERROR)

Правильное значение LEVEL может указывать только на:

•FATAL
•ALERT
•CRIT
•ERROR
•WARN
•NOTICE
•INFO
•DEBUG
•TRACE

Если нужно увидеть все записи, то нужно указать TRACE.

Есть два параметра в конфигурации, которые можно использовать в конфигурации каждого контейнера.

•lxc.logfile
•lxc.loglevel

Стоит помнить, что если в параметрах командной строки используются -o или -l, то они перекрывают значения в конфиге контейнера. Если хочется отправить отчёт об ошибке LXC, то хорошей идей будет приложить журнал работы с контейнером с —logpriority=DEBUG.

Отладка API.

Когда отлаживается код, использующий API LXC, то хорошей практикой является переписывания проблемного кода на C, использующий liblxc напрямую. Это поможет исключить проблему с биндингами (binding) и получить более чистую трассировку стека (stack traces) и упростить отчёт об ошибке.

Так же полезно выставить lxc.loglevel=DEBUG через set_config_item и увидеть, что происходит не правильно.

Тестирование.

До погружения в глубины отладки вашего кода, желательно проверить работу LXC на вашей машине. Стоит запустить lxc-checkconfig для проверки и понять: *а что ваше ядро поддерживает из возможностей LXC?* Если всё хорошо и вывод lxc-checkconfig обнадёживает, то можно перейти к тестам. Вам нужно скомпилировать и установить их, если вы ставили LXC из исходников. В Убунту есть пакет lxc-tests, который можно установить и получить весь набор тестов. Большинство тестов

исходит из предположения, что у вас установлена Ubuntu, но другие дистрибутивы линукс должны работать с тестами нормально. Если это не так, то шлите патчи.

Запускайте от root'а тесты lxc-test-* и обращайте внимание на неисправности. К сожалению, тест может оставить после себя хлам, который сам по себе может помешать следующему тесту. Поэтому нужно внимание человека и подчищайте после тестов.

Сообщение об ошибке.

Основной баг трекер LXC - https://github.com/lxc/lxc/issues

Отчёт об ошибке можно подать в рамках вашего дистрибутива или в upstream или туда и туда и связать их.

Для Ubuntu баг трекер находится по адресу https://bugs.launchpad.net/ubuntu/+source/lxc

Если вы проделали определённую работу по созданию отчёта, то можно обратиться к разработчикам в их почтовой рассылке, которые найдёте чуть ниже.

Отправка патча.

Разработчики LXC рады любому вкладу и счастливы сообщить, что насчитывают активных разработчиков около 60 человек, которые способствовали выходу LXC 1.0. Есть не так много правил, регулирующих вклад в работу. Требуется лишь указание лицензии, под которой вы публикуете патч, и что вы являетесь владельцем авторских прав на ваш код.

Что касается лицензирования, то всё что попадает в liblxc или в биндинги должно быть под лицензией LGPLv2.1+ или совместимой с ней, не налагая дополнительные ограничения. Бинарники и скрипты могут быть под лицензией LGLPv2.1+ или GPLv2. Если есть какие-либо сомнения, то выбор LGPLv2.1+ будет идеален.

Патчи можно отправить двумя способами:

•Отправив по почте lxc-devel@lists.linuxcontainers.org, например через git send-email
•Сделав pull request в github проекта.

Контакты.

Основной способ общения у разработчиков LXC - это почтовые рассылки. Их две.

Одна для вопросов от разработчиков и одна для пользователей:

•lxc-devel: https://lists.linuxcontainers.org/listinfo/lxc-devel
•lxc-users: https://lists.linuxcontainers.org/listinfo/lxc-users

Для более динамичного и живого общения есть канал IRC #lxcontainers на irc.freenode.net.

Последние замечания.

Цикл из 10 статей закончился до финального выхода LXC 1.0. Сейчас проект переходит из rc3 в rc4 и выход релиза стоит ожидать 20 февраля вечером или 21 февраля утром.

Данный цикл статей от Стефана Грабера может стать полезным справочным материалом для людей, осваивающих LXC 1.0.

Добавляя от себя пять копеек, хочется рассказать о свои граблях. Технология LXC для доступа виртуальных машин использует bridge lxcbr0. Я использовал bind9 вместо дефолтного dnsmasq, который используется в LXC для работы с DNS и DHCP. Пришлось вернуться к дефолтному dnsmasq. Заодно пришлось сменить дефолтное использование подсети 10.0.3.x, так как у меня на работе есть такая работающая подсеть.

Следует знать, что система Docker использует под капотом систему виртуализации на уровне операционной системы LXC, поэтому изучение LXC поможет лучше понять, что делает Докер.

Инструмент оркестровки Juju умеет работать не только с облачными образами, но и с "локальным" LXC. Благодаря этому факту, можно поиграть в облачного админа, используя LXC. Лично я хочу посветить этому вопросу время и написать в будущем статью об успехах или неудачах на этом поле.

Кстати, Unity Greeter умеет входить не только в локальную систему, но и в удалённые облачные инстансы, а так же в локальные контейнеры LXC. То есть при старте вашей системы, можно залогиниться в операционную систему контейнера. Всё это так же планирую попробовать в живую и предоставить вам результаты.

На этом пока всё! Удачи и успехов!